

Semester (Term, Year)	W2026
Course Code	AER 404
Course Section	4
Course Title	Introduction to Aerospace Engineering
Course Instructor	Reza Faieghi
Submission	Report
Submission No.	1
Submission Due Date	04/05/2026
Title	Project 2 Final Report
Submission Date	04/04/2026

Submission by (Name):	Student ID (XXXX1234)	Signature
*****	XXXX1234	P.T
*****	XXXX1234	G.M.
Adam Ling	501287341	A.L
*****	XXXX1234	S.M
*****	XXXX1234	S.R

By signing the above you attest that you have contributed to this submission and confirm that all work you contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, and "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Academic

Table of Contents

<i>Table of Contents</i>	2
<i>List of Figures</i>	3
<i>List of Tables</i>	4
<i>Introduction</i>	5
<i>Revised Problem Statement</i>	7
<i>Objectives List</i>	8
<i>Contributors and Task Allocation</i>	9
<i>Overall Software Architecture</i>	11
<i>Subsystem Design and Implementation</i>	14
<i>Simulation and Validation Results</i>	25
<i>Discussion</i>	27
<i>Conclusion</i>	28
<i>References</i>	29
<i>Appendix</i>	30

List of Figures

<i>Figure. 1</i>	A high-level illustration regarding software architecture	11
<i>Figure. 2</i>	Validation test results for the force equations module.	20
<i>Figure. 3</i>	First validation test for the moment equations module, no moments set	20
<i>Figure. 4</i>	Second validation test for moment equations module, one non-zero value set	21
<i>Figure. 5</i>	Standalone system architecture	22
<i>Figure 6</i>	The kinematic and navigation equation subsystems on simulink	24
<i>Figure 7</i>	Time responses of the aircraft for the given conditions	25

List of Tables

Table 1 Contributions and Corresponding Team Members 9

Table 2

Table 3

Table 4

Table 5

Table 6

Introduction

An aircraft manufacturing company is currently in the process of designing a new airliner. This new narrow-body twin-jet aircraft is designed to actively compete against Boeing and Airbus aircraft, including the Boeing 737 and the Airbus A320 series. Because of the unique design, there is currently no flight simulator accurate enough to model the twin-jet aircraft. Thus, the goal of this project is to develop a flight simulator for the purpose of pilot training. Of the three main pillars for designing the flight simulator, the most relevant one for our context is the physics engine, which models the flight dynamics in a virtual environment that emulates real-world flight behaviour. Because of the inherent complexity and scale of this project, the main focus is on subsystem design, systems integration, and system validation for success.

As part of the physics engine team, the primary task is to model the aircraft's behaviour, which includes various flight parameters that should be modelled in such a way that they respond accordingly to the pilot's inputs in a six-degree-of-freedom environment. The software of choice for modelling the simulator is through a MATLAB-based Simulink software, using the given MATLAB scripts for aerodynamic tendencies and aircraft limits. Because of the number of MATLAB modules, many of these inputs and resulting outputs have to be processed systematically and reliably, a difficult design issue given the complexity. The solution is given in the project brief and involves using specific numerical methods to accurately model complex flight dynamics. To receive inputs, the simulator will integrate with externally given systems, including FlightGear and a joystick.

The report order is outlined in the table of contents and begins with the revised problem statement to give adequate context, followed by objectives and work allocation to highlight the general workflow. After an overall overview of the software architecture, there will be enough

context to move forward with describing the subsystem design and implementation. This includes a detailed description of each subsystem and how each one receives inputs, processes data, and outputs the resulting data to other modules. The resulting simulations and results are presented and discussed, and the strengths, weaknesses, and future improvements are outlined in the conclusion.

Revised Problem Statement

A MATLAB-based Simulink software must be designed to reliably model six-degree-of-freedom flight concerning a newly designed narrow-body jet for pilot training. The software should fulfill the primary objective of receiving user inputs and accurately depicting the change in aircraft behaviour based on such inputs. Examples of user inputs include throttle settings regarding the engine and actuation of main control surfaces (aileron, elevator, and rudder). The inputs should be accurately processed using equations that can emulate all three Euler angles (roll, pitch, yaw), body rates, longitudinal, lateral, and vertical velocities of the aircraft, and geodetic coordinates throughout flight (latitude, longitude, and altitude). Naturally, these equations surround force, moments, kinematics, navigation, and geodetic coordinates. The specific equations are outlined in future sections.

There are various system constraints that guide our design in addition to our objectives. As previously mentioned, numerical analysis is limited to those relevant for flight mechanics, specifically the fourth-order Runge-Kutta methodology and flat-earth equations. In addition, the system is restricted to MATLAB-based Simulink software. When designing the software architecture, the system must be capable of integrating and communicating flawlessly with FlightGear and a joystick for real-time inputs from users. There are also MATLAB scripts and given Simulink templates that must also integrate with the designed software. These templates include the aircraft aerodynamic behaviour and parameters, alongside the intended Simulink workflow given by the TA.

Objectives List

1. Accurately model aircraft flight dynamics
 - 1.1. Model 6-DOF motion
 - 1.2. Represent forces and moments acting on the aircraft
 - 1.3. Include aerodynamic effects
 - 1.4. Include engine thrust effects
 - 1.5. Include gravitational effects
2. Respond correctly to pilot control inputs
 - 2.1. Throttle response for both engines
 - 2.2. Aileron, elevator, and rudder effects
 - 2.3. Realistic sensitivity to control inputs (Analog input to digital translation)
3. Provide complete aircraft state information
 - 3.1. Calculate velocity components (U, V, W)
 - 3.2. Calculate body rates (P, Q, R)
 - 3.3. Calculate orientation angles (roll, pitch, yaw)
 - 3.4. Calculate position (NED)
 - 3.5. Convert position to geodetic coordinates (GPS)
4. Stable simulation behaviour
 - 4.1. Physical consistency of results
 - 4.2. Numerical stability over time (no infinities)
 - 4.3. Smooth and continuous state evolution
5. Enable integration with external simulation systems
 - 5.1. With FlightGear for visualization
 - 5.2. With a joystick for real-time input
6. Maintain a modular and well-structured system design
 - 6.1. Divide the system into functional subsystems
 - 6.2. Readability
 - 6.3. Ease of debugging and testing
 - 6.4. Ease of switching variables

Contributors and Task Allocation

As a note, *all sections* except for the *Subsystem Design and Implementation* section were completed by Adam and Pravini. Subsystem Design and Implementation was a shared section among all group members to explain their respective parts:

Table 1. Contributions and Corresponding Team Members

Function	Purpose	Owner	Verifier 1	Verifier 2	Deadline
Input Conditioning	Provides different angular inputs from the joystick for aerodynamic coefficient calculations.	*****	*****	Adam	3/22/2026
Constants and Basic Variables	Provides the fundamental variables and constants that are necessary for further calculations.	*****	*****	Adam	3/24/2026
Aerodynamic Coefficients	Determines the strength of forces by converting the angles.	*****	*****	*****	3/25/2026
Aerodynamic Forces	Converts the aerodynamic coefficients into real forces on the aircraft.	*****	*****	*****	3/26/2026
Engine Forces	Calculates thrust in the X axis ($Y, Z = 0$) using throttle angles and mass.	*****	*****	*****	3/26/2026
Aerodynamic Moments	Computes the rotational forces based on the aerodynamic forces and coefficients.	*****	*****	*****	3/27/2026

Engine Moments	Finds the rotational effects provided by the engines through basic variables and thrust.	*****	*****	*****	3/27/2026
Force Equations	Calculates acceleration based on forces and thrust, which changes velocities as a result.	Adam	Adam	Adam	3/31/2026
Moment Equations	Calculates rotational acceleration and updates P, Q, and R.	Adam	Adam	Adam	3/31/2026
Navigation Equation	Changes the velocities into position coordinates on earth from force and kinematic equations.	*****	*****	*****	3/31/2026
Kinematic Equations	Calculates roll, pitch and yaw angles (orientation) from angular velocities.	*****	*****	*****	3/31/2026
Geodetic Coordinates	Converts NED coordinates to geodetic for GPS.	*****	*****	*****	4/1/2026
Connecting to FC	Uses the geodetic coordinates to pinpoint aircraft location on earth.	*****	*****	*****	4/2/2026

Overall Software Architecture

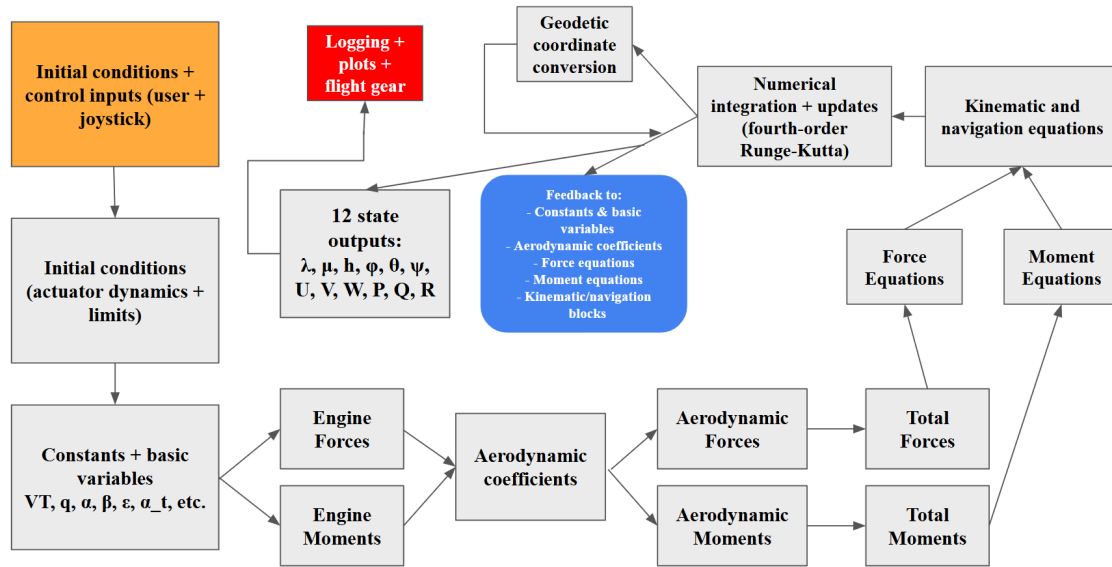


Figure 1. A high-level illustration regarding software architecture

At the beginning of the simulation (or before the simulation's start), the software receives position, altitude, attitude angles, velocities, body rates, etc. These inputs are intended to come from controls (e.g., throttle and control surface angle) and are managed through actuator limits (rate limits to make sure that control surfaces respond realistically and do not have instantaneous responses). After the initial step, the software will identify the constants and variables to be used in the later simulations. Some of these parameters are the mass of the aircraft, the inertia, the aerodynamic dimensions of the aircraft, air density, gravity, location of reference points, velocity, pressure, angle of attack (AOA), side slip angle, angle of downwash, and tail AOA. This block of code is typically viewed as a preparatory stage for the rest of the computations made in the simulation.

The thrust forces are produced by applying the prescribed variable inputs to the module of the engine's thrust force. The thrust forces produced are considered the total force applied to

the aircraft as a function of throttle inputs and characteristics of the aircraft along the x-y plane only as pertained to propulsive forces like flight inputs, etc; this does not include aerodynamic forces. The module of engine moments looks at how the forces produced by propulsion affect the rotation of the aircraft about its centre of mass. It does this by using the engine position relative to the centre of mass, which enables accurate modelling of roll, pitch, and yaw. The aerodynamic coefficient module calculates the aerodynamic coefficients based on the results of calculations made during previous modules. Therefore, the aerodynamic coefficients produced by the aircraft's behaviour in relation to the aerodynamics are inputs for the next module and will have an effect on future lift, drag, crosswind forces and moment calculations for rolling, pitching and yawing.

The aerodynamic force and moment module then simultaneously uses the previously sorted aerodynamic coefficients to determine forces resulting from pressure and geometry, while also calculating the moments based on the current aircraft configuration. Finally, the contributions each subsystem has made are summed to provide a total force and moment value, which segway into the kinematic and navigation equation modules to describe P, Q, R, roll angle, pitch angle, and yaw angle. In doing so, it effectively describes the body rates to develop the resulting orientation and position. The outputs from these subsystems are derivatives and must be integrated to provide proper final values. The methods of integration are the fourth-order Runge–Kutta values. At the same time, the position of the aircraft is not in any particularly useful form; the coordinates must be converted into proper longitude, latitude, and altitude values for proper integration with the external systems (FlightGear, graphics, and user interface systems).

Finally, the 12 key variables are used as an output to be stored, plotted, and used to interact with FlightGear. The immediate results after integration, however, are looped back to their respective subsystems for the process to start all over again. In the simplified diagram above, the arrows that would represent these transitions are replaced with a single block that simply states the loopback to avoid unnecessary clutter in a high-level diagram.

Subsystem Design and Implementation

Basic Variables Module

The basic variables subsystem computes the parameters required for the upcoming aerodynamic and force calculations. The module receives velocity components U, V, W as inputs and outputs the total magnitude of airspeed V_T , angle of attack α , sideslip angle β and dynamic pressure q using the following equations

$$V_T = (U^2 + V^2 + W^2)^{0.5}$$

$$\alpha = \text{atan}(W/U)$$

$$\beta = \text{asin}(V/V_T)$$

$$q = 0.5\rho V_T^2$$

Where $\rho=1.225\text{kg/m}^3$ is the air density.

Tail Effects Module

The tail effects subsystem is in charge of calculating the effects of air on the tail by calculating the downwash angle and the tail's angle of attack. This subsystem takes angle of attack α , pitch rate Q, total velocity V_T , and elevator deflection δ_E as inputs and provides downwash angle of the wings ϵ , and AoA of tail α_t using these equations

$$\epsilon = 0.25(\alpha - \alpha_0)$$

$$\alpha_t = \alpha - \epsilon + 1.3l_t Q/V_T + \delta_E$$

Where $l_t = 24.8\text{m}$, $\alpha_0 = -11.5^\circ$

Aerodynamic Coefficients Module

The Aerodynamic Coefficients subsystem calculates the non-dimensional aerodynamic coefficients required for the later aerodynamic force and moment calculations. In the implemented Simulink model, this subsystem receives the inputs

$\alpha, \beta, \epsilon, \alpha_t, P, Q, R, V_T, \delta_A, \delta_E$ and δ_R , and produces the outputs C_L, C_D, C_C, C_l, C_m , and C_n .

These coefficients describe the aerodynamic behaviour of the aircraft and are passed to the downstream aerodynamic force and aerodynamic moment calculations. The subsystem therefore serves as the stage that converts flight-condition variables and control surface deflections into coefficient values used elsewhere in the model.

The lift coefficient is calculated as the sum of the wing-body lift coefficient and the tail lift coefficient. In the MATLAB function, the wing-body contribution is computed as

$$C_{L,wb} = 5.5(\alpha - \alpha_0)$$

And the tail contribution is computed as:

$$C_{L,t} = 3.1\left(\frac{S_t}{S}\right)\alpha_t$$

The total lift coefficient is then

$$C_L = C_{L,wb} + C_{L,t}$$

Where $S = 260, S_t = 64$ and $\alpha_0 = -11.5$ deg, converted into radians in the code. This implementation then combines the main wing-body and tail effects into one total lift coefficient.

The drag coefficient is implemented as

$$C_D = 0.13 + 0.07(5.5\alpha + 0.654)^2$$

Which introduces a nonlinear dependence on angle of attack. The crosswind coefficient is calculated as

$$C_C = -1.6\beta + 0.24\delta_R$$

Showing that the lateral aerodynamic response depends on the sideslip angle and rudder deflection.

The rolling moment coefficient is calculated in the subsystem as

$$C_l = -1.4\beta - 11\left(\frac{1}{V_T}\right)P + 5\left(\frac{1}{V_T}\right)R - 0.6\delta_A + 0.22\delta_R$$

Where $l = 6.6m$ is the characteristic length used in the model. This coefficient captures the effect of sideslip, roll rate, yaw rate, aileron deflection, and rudder deflection on rolling tendency.

The pitching moment coefficient is implemented in the MATLAB function as

$$C_m = -0.59 - 3.1\left(\frac{S l_t}{S l}\right)(\alpha - \epsilon) - 4.03\left(\frac{S l_t}{S l^2}\right)\left(\frac{l q}{V_T}\right) - 3.1\left(\frac{S l_t}{S l}\right)\delta_E$$

Where $l_t = 24.8m$. This expression models the effect of the aerodynamic state and elevator input on aircraft pitching tendency. This yawing moment coefficient is calculated as:

$$C_n = (1 - 3.8179\alpha)\beta + 1.7\left(\frac{l}{V_T}\right)P - 11.5\left(\frac{l}{V_T}\right)R - 0.63\delta_R$$

This describes the aerodynamic yawing response due to sideslip, roll rate, yaw rate, and rudder deflection.

In implementation terms, the subsystem uses fixed aircraft constants directly inside the MATLAB Function block, including S , S_{ℓ} , l , l_{ℓ} , and α_0 . This reduces the number of incoming signals and keeps the coefficient calculations self-contained within a single block. The subsystem output consists only of the six aerodynamic coefficients, which makes it easy to connect to downstream modules without additional signal processing. This modular approach improves

readability and makes the aerodynamic coefficient calculations easier to isolate during debugging and validation.

Aerodynamic Moments Subsystem

The Aerodynamic Moments subsystem calculates the aircraft's aerodynamic moments in two steps. First, using the pitch, roll, and yaw moment coefficients in the equations below, moments are calculated.

$$l'_A = \overline{qc}SC_l \mid m'_A = \overline{qc}SC_m \mid n'_A = \overline{qc}SC_n$$

Then these moments are put in a 3x1 matrix and added with the real aerodynamic forces vector from the Aerodynamic Forces subsystem.

$$\begin{pmatrix} l_A \\ m_A \\ n_A \end{pmatrix} = \begin{pmatrix} l'_A \\ m'_A \\ n'_A \end{pmatrix} + \begin{pmatrix} X_A \\ Y_A \\ Z_A \end{pmatrix} \times \begin{pmatrix} 0.11\overline{c} \\ 0 \\ 0.1\overline{c} \end{pmatrix}$$

The real aerodynamic forces are in a cross product with that extra vector to account for moments generated due to the aircraft's center of mass. This equation gives a vector of the actual aerodynamic moments.

Engine Moments Subsystem

The Engine Moment subsystem calculates the vector of the total rolling, pitching, and yawing moments of the engine. The equation is shown below:

$$\begin{pmatrix} l_{T,i} \\ m_{T,i} \\ n_{T,i} \end{pmatrix} = \begin{pmatrix} X_{T,i} \\ 0 \\ 0 \end{pmatrix} \times (cm - apt_i)$$

This equation must be run for each engine.

Geodetic Coordinates Subsystem

The Geodetic Coordinates Equations subsystem takes NED coordinates and converts them to GPS coordinates. Earth's equatorial radius (a), the first eccentricity (e), and phi (ϕ) are used to calculate the meridian radius (M) and the prime vertical radius (N) at a given location. This is shown in the equations below:

$$M = \frac{a(1-e^2)}{\sqrt{(1-e^2 \sin^2 \phi)^3}} \mid N = \frac{a}{\sqrt{1-e^2 \sin^2 \phi}}$$

These new values are used with NED coordinates to get GPS longitude (λ_{dot}) and latitude (μ_{dot}).

$$\mu_{dot} = \frac{p_{dot,N}}{M+h} \mid \lambda_{dot} = \frac{p_{dot,E}}{(N+h)\cos(\mu_{dot})}$$

The aircraft's motion is simulated using a set of force equations that describe the translational motion through U, V, and W velocity components over time. The subsystem was made independently and, as such, has five constant blocks that feed into a main MATLAB function block. The five constant blocks were used as placeholders for actual inputs that would be integrated into the code later. These constant blocks represent the aerodynamic forces, engine forces, body rates, a state vector [U, V, W, ϕ , θ], and a constant vector as a set of arrays. The MATLAB function block then processes these results by using equations in the supplementary material, which yield a derivative vector [U', V', W'] that was displayed through a scope function. To test the module before integration, a gravity-only case was used. The result from this test was an output of [0, 0, 9.81]. Therefore, the module functions properly.

Force and Moment Equations Module

The aerodynamic forces subsystem uses the calculated aerodynamic coefficients to calculate real forces acting on the aircraft in relation to the body. The subsystem uses lift coefficient C_L , drag coefficient C_D , crosswind coefficient C_C , dynamic pressure q , angles α and β to calculate forces real drag X_A , crosswind Y_A , lift Z_A in the FRD coordinate system, acting as a bridge between aerodynamic modelling and body dynamics, using these equations

$$D = qSC_D$$

$$C = qSC_C$$

$$L = qSC_L$$

$$X_A = -D \cos(\alpha) \cos(\beta) + C \cos(\alpha) \sin(\beta) + L \sin(\alpha)$$

$$Y_A = -D \sin(\beta) - C \cos(\beta)$$

$$Z_A = -D \sin(\alpha) \cos(\beta) + C \sin(\alpha) \sin(\beta) - L \cos(\alpha)$$

The engine forces subsystem calculates the thrust forces X_{T1} , X_{T2} given by the engines based on the throttle inputs δ_{T1} , δ_{T2} , using the following equations

$$X_{T1} = mg\delta_{T1}$$

$$X_{T2} = mg\delta_{T2}$$

$$X_T = X_{T1} + X_{T2}$$

Since the thrust is aligned with the engine orientation, $Y_T = Z_T = 0$.

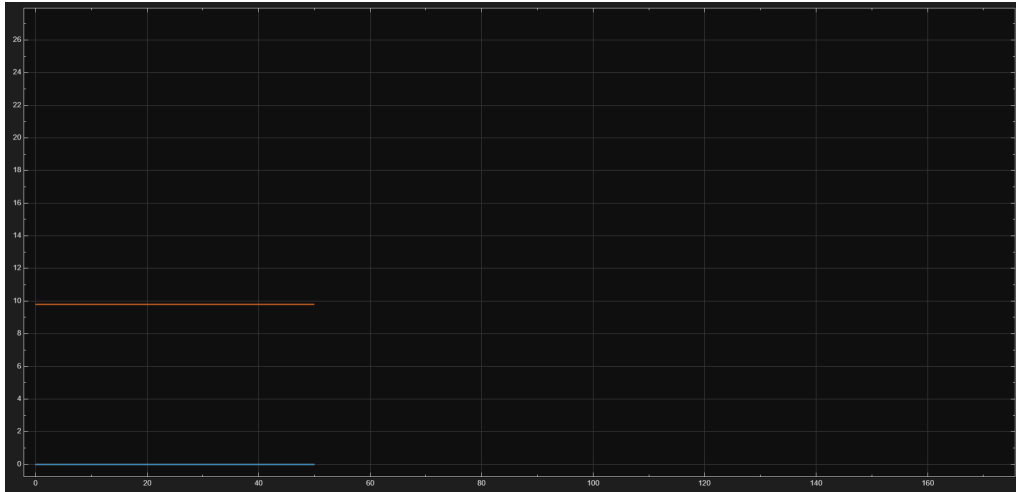


Figure 2. Validation test results for the force equations module.

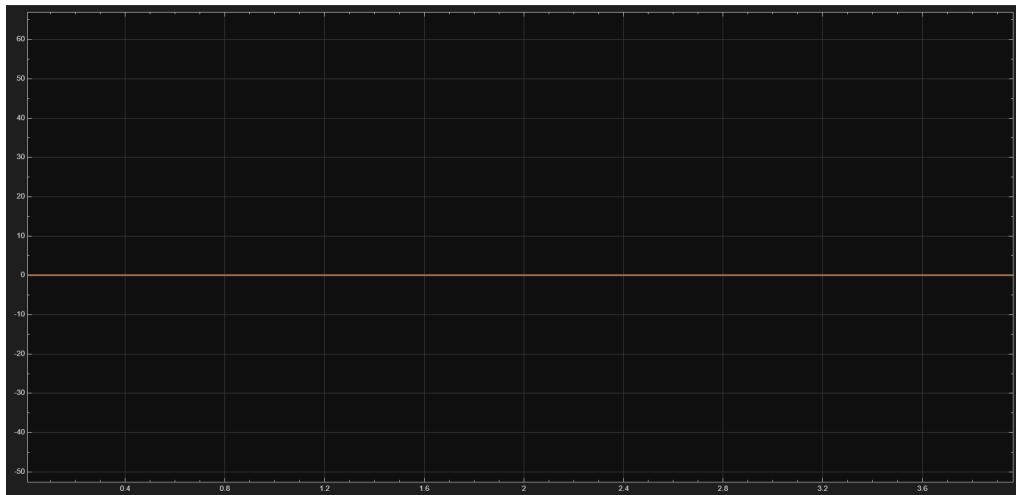


Figure 3. First validation test for the moment equations module, no moments set

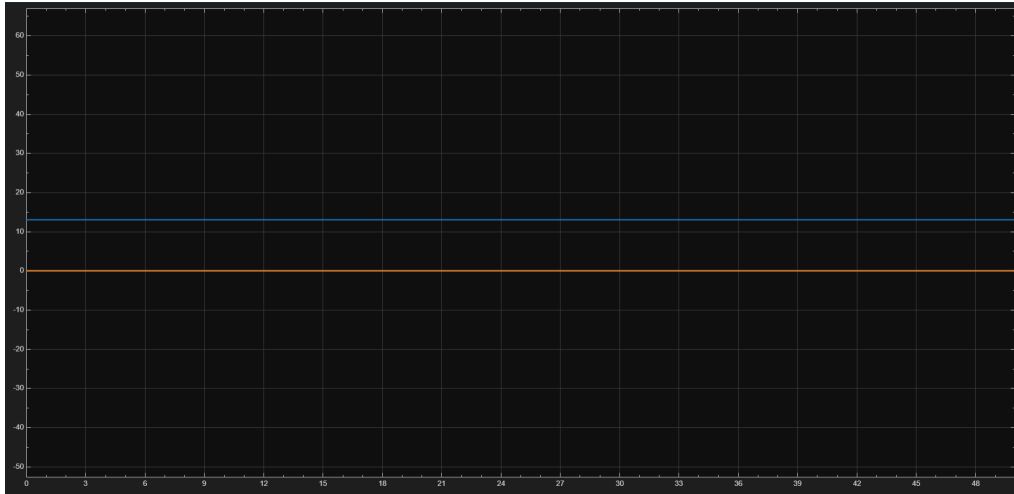


Figure 4. Second validation test for moment equations module, one non-zero value set

The moment equations module is similar to the force equation module in that constants were used as placeholders for future integration. This module is designed to mimic the aircraft's rotation tendencies. To do so, the module accepts the variables provided by the four constant blocks, which include the two main moment values, the body-rate vector $[P, Q, R]$, and the various J values. By using another MATLAB function block, the inputs are processed using equations from the supplementary, which yielded a derivative vector $[P', Q', R']$. To test the subsystem, two cases were used. A static case with a zero moment value yielded no angular accelerations, a good result. The second test was using a pitch moment only, which led to a single non-zero derivative that corresponded to the correct axis, a correct result.

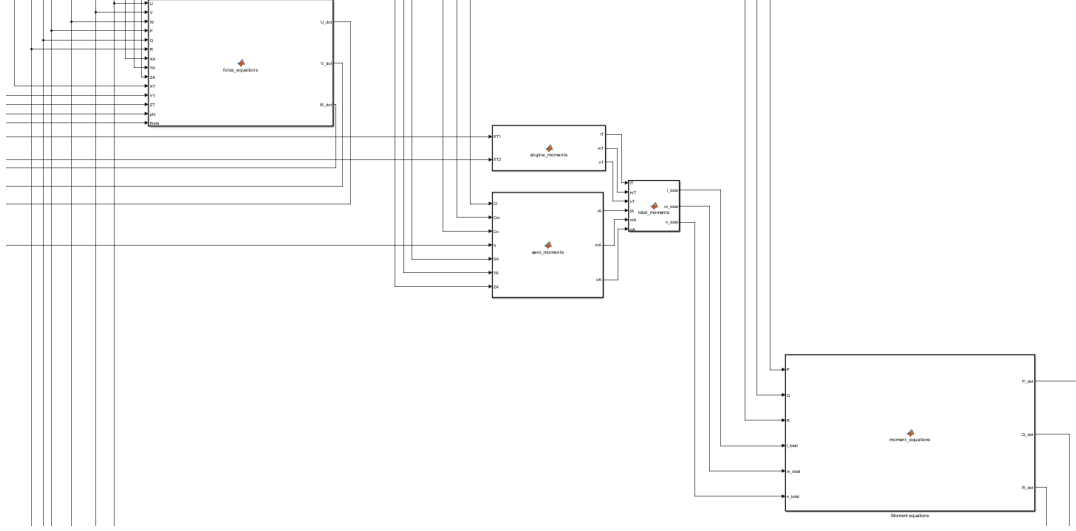


Figure 5. Standalone system architecture

Kinematics and Navigation Equations Module

The kinematic equations calculate the change in euler angles: yaw, pitch and roll angle using the angular rates. In other words, It specifically uses P, Q, R, phi, theta as inputs to generate phi_dot, theta_dot and psi_dot which indicate the orientation of the aircraft. These variables are connected to the subsystem using the signal lines.

$$\begin{aligned}\dot{\phi} &= P + \tan \theta (Q \sin \phi + R \cos \phi) \\ \dot{\theta} &= Q \cos \phi - R \sin \phi \\ \dot{\psi} &= \frac{Q \sin \phi + R \cos \phi}{\cos \theta}\end{aligned}$$

The roll rate orientation uses the roll rate, pitch and yaw rate as well as the roll and pitch angles. The change in pitch orientation equation uses the pitch, roll rates and the roll angle. Finally, the yaw rate orientation angle is calculated by using the roll, pitch angles and the yaw pitch rates.

The navigation equations translate the position with respect to time in the earth frame. Producing the velocity in the north, east directions and the altitude rate determining the

geographic position of the aircraft. This subsystem uses U, V, W, phi, theta and psi as the input variables and outputs pE_dot, pN_dot and h_dot. The subsystem is connected to these variables with the signal lines.

$$\begin{aligned}\dot{p}_N &= U \cos \theta \cos \psi + V(-\cos \phi \sin \psi + \sin \phi \sin \theta \cos \psi) + W(\sin \phi \sin \psi + \cos \phi \sin \theta \cos \psi), \\ \dot{p}_E &= U \cos \theta \sin \psi + V(\cos \phi \cos \psi + \sin \phi \sin \theta \sin \psi) + W(-\sin \phi \cos \psi + \cos \phi \sin \theta \sin \psi), \\ \dot{h} &= U \sin \theta - V \sin \phi \cos \theta - W \cos \phi \cos \theta.\end{aligned}$$

All the navigation equations above are calculated by using the body frame velocities: U,V, W and the euler angles.

The kinematic and navigation equations both play a significant role in translating the aircraft's body forces into actual position coordinates that can be obtained to track the location of the airplane.

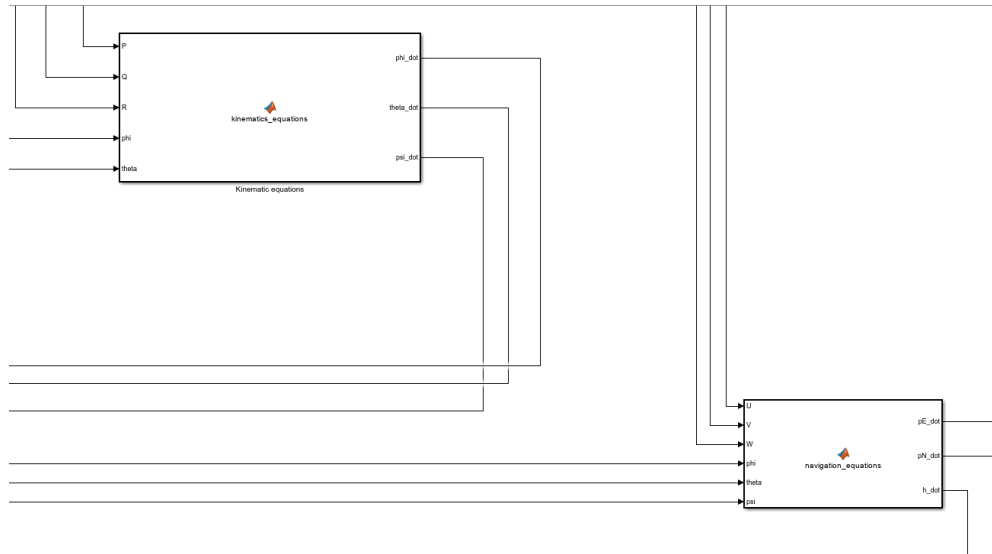


Figure 6. The kinematic and navigation equation subsystems on simulink

Simulation Results

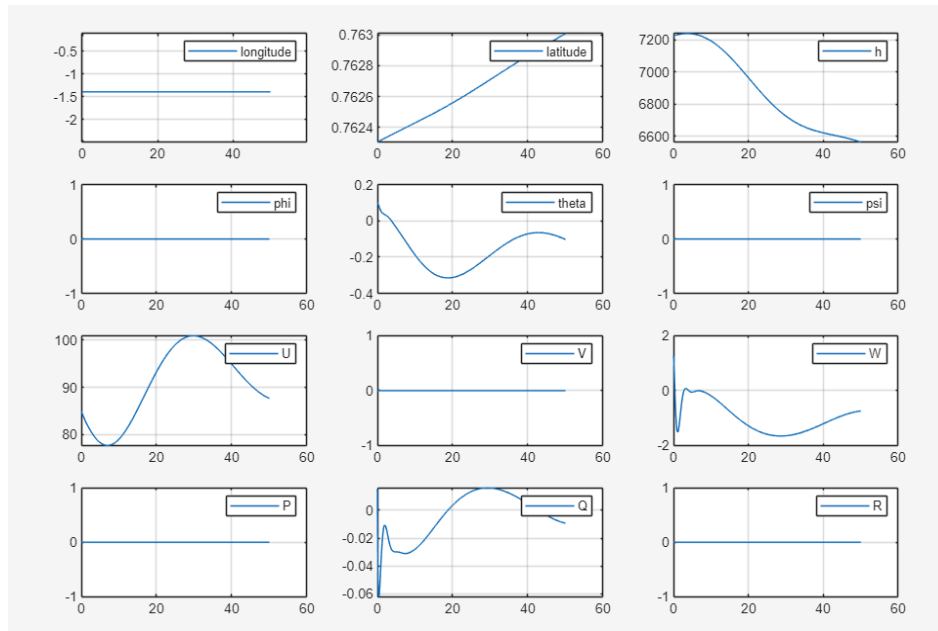


Figure 7. Time responses of the aircraft for the given conditions.

To validate the aircraft software, constant inputs were provided which produced the resulting plots above. Each graph is plotted for 50 seconds with the mentioned variable with respect to time. The longitude graph is constant throughout the 50 seconds of flight and slightly negative, indicating that the aircraft is flying slightly in the west direction. The latitude graph is increasing steadily meaning that the aircraft is moving in the north direction. This concludes that the aircraft is flying in the northwest direction on earth. The altitude (h) is portrayed as a decreasing graph showing that the aircraft is descending, in other words losing height. The phi graph shows the changes in the roll axis, indicating rotation about the x-axis. This graph is constant at zero which means the aircraft is not “rolling”. The theta graph visualizes the changes in the pitch axis. The nose of the airplane is tilted downward which matches the altitude graph and tilts up its nose slightly near the end to reach a stable position. The psi graph indicates the

changes in the yaw axis. It is shown to be constant and at zero so the aircraft is not turning in any direction and flying forward which matches the latitude graph. The U graph is the speed in the x-axis and it shows a slight deceleration, an acceleration until 30 seconds and a final deceleration until the end. The V graph is the velocity progress in the y direction which portrays a zero constant graph. Meaning no turning movements done by the aircraft. The W graph is the latitude speed and shows many changes. The aircraft first ascends, stays stable for a few seconds, descends until 30 seconds and ascends again. The P graph states how fast the aircraft rolls and since it is zero and constant it has no rolling motion. The Q graph shows the pitch rate of the aircraft and this graph shows major changes. The rate of movement of the nose cone at first increases quickly and later switches between a slow and fast tilt. The R graph shows the yaw rate of the graph which visualizes the turning rate of the aircraft. This shows a constant zero value, indicating no change and corresponds to the psi graph.

Discussion

The final system was made by using individual modules. Each module was designed to fulfill specific purposes, ranging from collecting inputs, processing inputs, and outputting the processed result. Each module was designed individually, and as such, underwent various tests to ensure correct performance before overall system integration. These tests were nominal, and the order that modules process data follows the following: initial conditions, aerodynamic, engine, force, moment, kinematic, navigation, geodetic calculations, and output/feedback loop. When examining the software from a more holistic point of view, it is apparent that the simulator completes all intended functions. User inputs regarding throttle and control surface changes were accurately received, processed, and output to the corresponding flight dynamic changes. The

system also integrated seamlessly with the FlightGear and joystick, which led the system to stretch beyond a simple physics engine, but a flight simulator capable of pilot training. Under MATLAB, the block code made it easy to keep track of progress, edit designs, and integrate everything successfully. This aircraft simulator executed successfully due to the set up of the system. The sets of equations were translated into MATLAB code inside a function with variables describing the purpose. Each equation was implemented within individual subsystems, where inputs and outputs were connected through the signal lines. The signal lines were linked neatly to simplify the debugging process. This is the main part of the aircraft process, which is placed inside another subsystem, later connected to the input and output of the overall system. Several issues were encountered while building this system, which were eventually solved. Since there were many files used, many system integration problems while connecting flight gear to the simulation were solved. Time was also spent debugging the system to provide the correct graphs similar to the given graphs.

Conclusion

In conclusion, the flight simulator was built in MATLAB, using the Simulink feature. By implementing the given fundamental aircraft equations such as aerodynamic forces, aerodynamic moments, engine forces, engine moments, force equations, moment equations, kinematic equations and navigation equations to calculate the characteristics of the aircraft. The aerodynamic forces convert the aerodynamic coefficients into forces on the aircraft. The aerodynamic moments were rotational moments based on the forces. The engine force equations are used to calculate the force in the x-direction. The moment equations calculate the rotational moments of the aircraft. The kinematic equations calculate the orientation of the body frame

while the navigation equations translate the position with time. The benefits of this design is that it can detect the exact and accurate position coordinates of the airplane. The signal lines in the simulink file are also neatly organized. The disadvantages of this design is that it uses many softwares to process the outputs. In addition, it is difficult to spot mistakes while using a software like simulink. In the future, this design can be improved by using a more organized program that can avoid causing errors.

References

- [1] R. Faieghi, “*Project 2 Supplementary Material*”, Department of Aerospace Engineering, Toronto Metropolitan University. [Accessed: March 21, 2026]. Project 2 Supplementary Material.

Appendix

Basic Variables Module

$$V_T = (U^2 + V^2 + W^2)^{0.5}$$

$$\alpha = \text{atan}(W/U)$$

$$\beta = \text{asin}(V/V_T)$$

$$q = 0.5\rho V_T^2$$

Tail Effects Module

$$\varepsilon = 0.25(\alpha - \alpha_0)$$

$$\alpha_t = \alpha - \varepsilon + 1.3l_t Q/V_T + \delta_E$$

Aerodynamic coefficient equations:

$$C_{L,wb} = 5.5(\alpha - \alpha_0)$$

$$C_{L,t} = 3.1\left(\frac{S_t}{S}\right)\alpha_t$$

$$C_L = C_{L,wb} + C_{L,t}$$

$$C_D = 0.13 + 0.07(5.5\alpha + 0.654)^2$$

$$C_C = -1.6\beta + 0.24\delta_R$$

$$C_l = -1.4\beta - 11\left(\frac{1}{V_T}\right)P + 5\left(\frac{1}{V_T}\right)R - 0.6\delta_A + 0.22\delta_R$$

$$C_m = -0.59 - 3.1\left(\frac{S_t l_t}{Sl}\right)(\alpha - \epsilon) - 4.03\left(\frac{S_t l_t}{Sl^2}\right)\left(\frac{lQ}{V_T}\right) - 3.1\left(\frac{S_t l_t}{Sl}\right)\delta_E$$

$$C_n = (1 - 3.8179\alpha)\beta + 1.7\left(\frac{l}{V_T}\right)P - 11.5\left(\frac{l}{V_T}\right)R - 0.63\delta_R$$

Aerodynamic Moments Subsystem

$$l'_A = \overline{qc}SC_l \mid m'_A = \overline{qc}SC_m \mid n'_A = \overline{qc}SC_n$$

$$\begin{pmatrix} l_A \\ m_A \\ n_A \end{pmatrix} = \begin{pmatrix} l'_A \\ m'_A \\ n'_A \end{pmatrix} + \begin{pmatrix} X_A \\ Y_A \\ Z_A \end{pmatrix} \times \begin{pmatrix} 0.11\overline{c} \\ 0 \\ 0.1\overline{c} \end{pmatrix}$$

Engine Moments Subsystem

$$\begin{pmatrix} l_{T,i} \\ m_{T,i} \\ n_{T,i} \end{pmatrix} = \begin{pmatrix} X_{T,i} \\ 0 \\ 0 \end{pmatrix} \times (cm - apt_i)$$

Geodetic Coordinates Subsystem

$$M = \frac{a(1-e^2)}{\sqrt{(1-e^2 \sin^2 \phi)^3}} \mid N = \frac{a}{\sqrt{1-e^2 \sin^2 \phi}}$$

$$\mu_{dot} = \frac{p_{dot,N}}{M+h} \mid \lambda_{dot} = \frac{p_{dot,E}}{(N+h)\cos(\mu_{dot})}$$

Force and Moment equations:

$$D = qSC_D$$

$$C = qSC_C$$

$$L = qSC_L$$

$$X_A = -D \cos(\alpha) \cos(\beta) + C \cos(\alpha) \sin(\beta) + L \sin(\alpha)$$

$$Y_A = -D \sin(\beta) - C \cos(\beta)$$

$$Z_A = -D \sin(\alpha) \cos(\beta) + C \sin(\alpha) \sin(\beta) - L \cos(\alpha)$$

$$X_{T1} = mg\delta_{T1}$$

$$X_{T2} = mg\delta_{T2}$$

$$X_T = X_{T1} + X_{T2}$$

$$J = \begin{pmatrix} J_x & J_{xy} & J_{xz} \\ J_{xy} & J_y & J_{yz} \\ J_{xz} & J_{yz} & J_z \end{pmatrix}. \quad \begin{aligned} \dot{U} &= RV - QW - g \sin \theta + \frac{X_T + X_A}{m}, \\ \dot{V} &= -RU + PW + g \sin \phi \cos \theta + \frac{Y_T + Y_A}{m}, \\ \dot{W} &= QU - PV + g \cos \phi \cos \theta + \frac{Z_T + Z_A}{m}. \end{aligned}$$

$$\begin{aligned}
\Gamma \dot{P} &= J_{xz}[J_x - J_y + J_z]PQ - [J_z(J_z - J_y) + J_{xz}^2]QR + J_z(l_A + l_T) + J_{xz}(n_A + n_T), \\
J_y \dot{Q} &= (J_z - J_x)PR - J_{xz}(P^2 - R^2) + m_A + m_T, \\
\Gamma \dot{R} &= [(J_x - J_y)J_x + J_{xz}^2]PQ - J_{xz}[J_x - J_y + J_z]QR + J_{xz}(l_A + l_T) + J_x(n_A + n_T).
\end{aligned}$$

Kinematics and Navigation Equations:

$$\begin{aligned}
\dot{p}_N &= U \cos \theta \cos \psi + V(-\cos \phi \sin \psi + \sin \phi \sin \theta \cos \psi) + W(\sin \phi \sin \psi + \cos \phi \sin \theta \cos \psi), \\
\dot{p}_E &= U \cos \theta \sin \psi + V(\cos \phi \cos \psi + \sin \phi \sin \theta \sin \psi) + W(-\sin \phi \cos \psi + \cos \phi \sin \theta \sin \psi), \\
\dot{h} &= U \sin \theta - V \sin \phi \cos \theta - W \cos \phi \cos \theta.
\end{aligned}$$